DLX-NN: A DLX RISC Processor with a Neural-Network Activation Engine Extension

Project Number: 3372

Project Report

Student: Saed Abu Fool ID: 213851579

Student: Saleh Khalil ID: 213310485

Supervisor: **Oren Ganon**

Project Carried Out at: Tel Aviv University

Contents

List of Figures

List of Tables

List of Equations

Abstract

Neural-network inference depends on non-linear activation functions: Sigmoid, Tanh, GELU, ReLU and Softmax. On a small embedded processor these are expensive in software, since each call expands into tens of instructions on a multi-cycle machine. This project extends the 32-bit multi-cycle simplified DLX processor with a hardware *neural-network activation engine* that computes all five functions as single custom instructions; the extended core is referred to below as DLX-NN. All arithmetic uses the Q16.16 signed fixed-point format[1].

The design goal was to accelerate inference while keeping silicon cost minimal. The three sigmoid-based functions (Sigmoid, Tanh, GELU) share a single piecewise-linear sigmoid core whose slopes are powers of two: Tanh and GELU are folded onto it through the identities $\tanh(x) = 2\sigma(2x) - 1$ and $\mathrm{GELU}(x) \approx x \cdot \sigma(1.702x)$, so the scalar activation path contains no general-purpose multipliers in its critical path. ReLU is a sign-bit test. Softmax, a vector function, is implemented as a DMA-driven engine that streams the operand vector through the external SRAM behind a new 2:1 bus arbiter, using a two-pass online-softmax algorithm with a multiplier-free exponent unit and a piecewise-linear reciprocal. The DLX control FSM grows from 21 to 25 states and the instruction set gains five opcodes.

The project delivers synthesizable Verilog RTL of the extended processor, the five new instructions integrated into the RESA lab assembler, Python golden models with SystemVerilog testbenches that verify the hardware bit-exactly, and an FPGA implementation running on a Xilinx Spartan-6 in the laboratory RESA monitor environment. Measured accuracy meets the quantitative requirement (mean squared error ≤ 0.025) with large margin: the maximum absolute sigmoid error is 3.19×10^{-4}, and over a 1050-case sweep the softmax MSE averages 1.03×10^{-4} with a worst case of 8.6×10^{-3}.

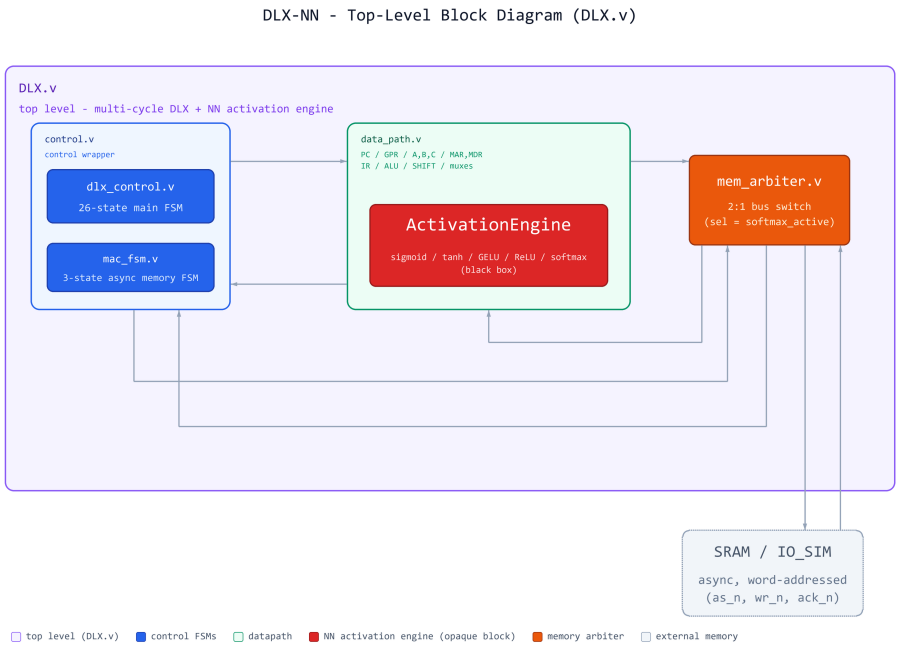

Figure 1: DLX top-level schematic.

[1] Q16.16: 32-bit two's-complement fixed point with 16 integer and 16 fractional bits; resolution $2^{-16} \approx 1.5 \times 10^{-5}$.

1 Introduction

1.1 Project Goals

The goal of this project is to extend a 32-bit multi-cycle DLX processor with a hardware *neural-network activation engine*, so that the five activation functions most common in neural-network inference (Sigmoid, Tanh, GELU, ReLU and Softmax) execute as single custom instructions instead of long software routines. The extended processor ("DLX-NN") must remain a fully functional DLX: every base instruction keeps its semantics, and the extension is exercised through five new opcodes.

The quantitative targets defined for the project are:

- **Latency:** $\geq 30\%$ reduction in clock cycles per activation evaluation compared to an equivalent software implementation running on the same DLX.
- **Accuracy:** mean squared error (MSE) < 0.025 against both the exact mathematical function and the bit-accurate golden model.
- **Area:** FPGA resource overhead of the extension $\leq 20\%$ (LUTs) relative to the baseline DLX implementation.

1.2 Motivation

Activation functions sit between the linear layers of a neural network and are evaluated millions of times during inference. They are what makes deep models expressive, and they are exactly what a minimal RISC core is worst at. The base DLX supports 32-bit integers and the basic operations: add, subtract, shift, compare and logic. It has no multiplier, no way to evaluate e^x, and cannot even represent fractional values. A function such as $\sigma(x) = 1/(1 + e^{-x})$ must therefore be emulated with long instruction sequences: our software PWL sigmoid takes roughly 40 instructions per call, and softmax adds loops, running maxima and a division per element, repeated for every neuron in every layer. For edge devices that run inference on-core, this per-activation cost is a real bottleneck.

A hardware activation engine attacks exactly this cost, and activation functions happen to be ideal candidates for cheap fixed-point approximation: they feed further layers that are themselves noisy, so moderate approximation error is tolerable, and piecewise-linear segments with power-of-two slopes need no multiplier at all.

There is also a point to making the host a DLX. It is a classic teaching processor, about as simple as a working RISC machine gets, and extending it with native activation instructions shows that neural-network inference is not reserved for large or specialized cores. The project covers the complete path: choosing hardware-friendly approximations from the literature, implementing them in Verilog, integrating them into a real processor's datapath, control FSM and memory system, and verifying them against bit-accurate golden models.

1.3 Solution Approach

The design follows four guiding decisions:

1. **One number format everywhere.** All activation arithmetic uses signed Q16.16 fixed-point (Section 2.2): 32-bit words with 16 fractional bits, matching the DLX

word size, so the integer datapath carries fractional values without any floating-point hardware.

2. **Multiplier-free critical paths.** Sigmoid uses a 5-segment piecewise-linear approximation whose slopes are all powers of two, so every multiply reduces to an arithmetic shift. The softmax exponential is a shift-and-add approximation; its reciprocal is an 8-segment PWL. Where a true multiply is unavoidable (GELU post-scaling, softmax normalization), the operands are bit-narrowed and the multiply is isolated in its own pipeline stage.
3. **Resource sharing.** Tanh and GELU are computed through the sigmoid core via $\tanh(x) = 2\sigma(2x) - 1$ and $\mathrm{GELU}(x) \approx x\,\sigma(1.702x)$, so a single PWL core serves three instructions; the two derived functions cost only multiplexers and a little pre- and post-processing (Section 4.1.3).
4. **Scalar vs. vector split.** The four scalar functions read one register and write one register, fitting the existing execute/write-back pattern; they run in an activation engine beside the ALU, under new states in the control FSM. Softmax is a vector function, since every output depends on all inputs, so it is implemented as a memory-to-memory DMA engine that temporarily takes over the SRAM bus through a new arbiter while the CPU waits (Section 4.1.5).

Each unit was written against a Python golden model and checked bit-exact in SystemVerilog simulation before synthesis for the FPGA.

1.4 Comparison with Existing Work

Hardware activation approximation is a well-studied area. Two published methods were adopted directly, and the third point below is what this project adds on top of them:

- **PLAN-style piecewise-linear approximation.** Pan et al. [1] restrict the segment slopes to powers of two, so every multiply collapses into an arithmetic shift and only a handful of slope/bias pairs must be stored, unlike LUT/ROM tabulation or CORDIC and Taylor iteration. We adopt this method for the sigmoid core and add a Taylor "repair" segment around $x = 1$ that cuts the maximum sigmoid error by two orders of magnitude (Section 2.4).
- **Online softmax.** Conventional softmax hardware pipelines three passes (max, sum, divide) or instantiates a large divider. We follow the online two-pass formulation of Hirayae et al. [2] (ICECS 2025), which fuses the running maximum and a rescaled denominator into a single pass and replaces both e^x and $1/d$ with shift and PWL approximations.
- **Full processor integration.** Both papers describe standalone units. Here the units live inside a real DLX's decode/execute/write-back flow, share its memory system through a new bus arbiter, and are exercised end-to-end by assembly programs assembled with the RESA lab toolchain and run on the FPGA. Tanh and GELU are derived from the same sigmoid core through identities rather than built as dedicated hardware, which keeps the added area close to zero.

2 Theoretical Background

2.1 The DLX Processor

DLX is a classic teaching RISC architecture: a 32-bit load/store machine with 32 general purpose registers (R0 hard-wired to zero, R31 used as the link register), three instruction formats (R-type, I-type) and a small orthogonal instruction set. The variant used in this project ("simple DLX") follows the laboratory specification [3] and is *multi-cycle, non-pipelined*: a central finite-state machine walks every instruction through fetch, decode, execute and write-back states, so each instruction takes several clock cycles but the datapath hardware is minimal and every architectural register (PC, IR, A, B, C, MAR, MDR) is explicit.

2.2 Q16.16 Fixed-Point Representation

All activation arithmetic uses signed Q16.16 fixed-point: a 32-bit two's complement integer interpreted with 16 fraction bits (Figure 2). A word w (interpreted as a signed integer) represents the real value

$$x = \frac{w}{2^{16}}, \qquad x \in [-32768,\ +32767.99998], \qquad \text{resolution } 2^{-16} \approx 1.53 \times 10^{-5}. \tag{1}$$

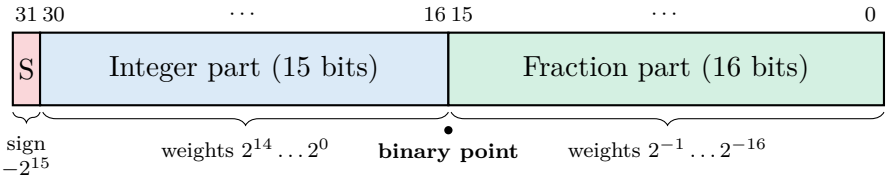

Figure 2: Q16.16 signed fixed-point bit-field layout.

Addition and subtraction are plain 32-bit two's complement operations. Multiplication of two Q16.16 values produces a 64-bit Q32.32 product; the Q16.16 result is the bit-slice [47:16] of that product. Two implementation rules follow directly and are used consistently in the RTL: (i) signed operands must be wrapped in `$signed()` (or declared signed) so that right shifts are *arithmetic* ($\ggg$) and products sign-extend correctly; (ii) a multiplication by a constant that is a sum of a few powers of two can be replaced by a shift-add tree, e.g. $1.702x \approx x + x/2 + x/8 + x/16 + x/64 + x/256 = 1.70703125\,x$.

2.3 Activation Functions in Neural Networks

A neural-network layer computes a linear map followed by an element-wise non-linearity. The five functions implemented in this project cover the common cases:

$$sigmoid(x) = \frac{1}{1+e^{-x}}, \qquad \tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}, \qquad \mathrm{ReLU}(x) = \max(0, x), \tag{2}$$

$$\mathrm{GELU}(x) = x \cdot \Phi(x) \tag{3}$$

where Φ is the standard normal CDF. Softmax normalizes a *vector* $x_{1..N}$ into a probability distribution; the numerically stable form subtracts the maximum $m = \max_i x_i$ before exponentiation:

$$y_j = \frac{e^{x_j - m}}{\sum_{i=1}^{N} e^{x_i - m}}, \qquad j = 1, \dots, N. \tag{4}$$

Sigmoid, Tanh, GELU and ReLU are scalar (one input, one output) and map naturally onto a register-to-register instruction. Softmax couples all N elements through the denominator, which is why it receives a fundamentally different, memory-streaming implementation.

2.4 Piecewise-Linear Approximation of the Sigmoid

The sigmoid core follows the PLAN methodology of Pan et al. [1]: $\sigma(x)$ is approximated for $x \geq 0$ by a handful of linear segments $y = a_k x + b_k$ whose slopes a_k are restricted to powers of two, so each segment reduces to one arithmetic shift and one addition, with no multiplier. Negative inputs reuse the symmetry $\sigma(-x) = 1 - \sigma(x)$, so only $|x|$ is ever evaluated. The five segments used in this project are

$$\sigma(x) \approx \begin{cases} 1, & x \geq 5 \\ \frac{1}{32}x + 0.84375, & 2.375 \leq x < 5 \\ \frac{1}{8}x + 0.625, & 1.125 \leq x < 2.375 \\ \frac{1}{8}x + 0.60606, & 0.875 \leq x < 1.125 \\ \frac{1}{4}x + 0.5, & 0 \leq x < 0.875 \end{cases} \tag{5}$$

We have added another segment(the fourth segment) to get better results.

2.5 Tanh and GELU through the Sigmoid Core

Tanh and GELU fold onto the same sigmoid core through the identities

$$\tanh(x) = 2\,\sigma(2x) - 1, \qquad \mathrm{GELU}(x) \approx x \cdot \sigma(1.702x). \tag{6}$$

Neither pre-scaling costs much: $2x$ is a left shift, and $1.702x$ is the shift-add tree from Section 2.2. Tanh post-processing is one left shift and an add of -1. GELU post-processing is the one point in the scalar engine that needs a true multiplier, for the final $x \cdot \sigma$ product.

2.6 Hardware-Oriented Online Softmax

Softmax follows Algorithm 3 of Hirayae et al. [2], a two-pass *online* formulation aimed at hardware. Pass 1 keeps a running maximum m_i and a running denominator d_i, rescaling d_i whenever the maximum grows, so max-finding and accumulation fuse into a single sweep:

$$m_i = \max(m_{i-1}, x_i), \qquad d_i = d_{i-1} \cdot \mathrm{shift\_exp}(m_{i-1} - m_i) \; + \; \mathrm{shift\_exp}(x_i - m_i), \tag{7}$$

Pass 2 then emits each output with a single multiply: $y_j = \mathrm{shift\_exp}(x_j - m_N) \cdot \mathrm{pwl\_recip}(d_N)$. Algorithm 1 shows the full flow as run by the RTL.

Both approximated primitives also come from [2]:

shift_exp. For $x \leq 0$, starting from $e^x = 2^{x \log_2 e}$ with the shift-friendly approximations $\log_2 e \approx 1.5$ and $\ln 2 \approx 0.5$ (paper eq. 1):

$$v = 1.5x = x + (x \ggg 1), \quad e^x \approx \left(1 + \tfrac{1}{2}\,\mathrm{frac}(v)\right) \gg |\lfloor v \rfloor|\,, \tag{8}$$

pwl_reciprocal. $1/d$ is approximated by eight linear segments aligned to the power-of-two intervals $[2^k, 2^{k+1})$, $k = 0..7$ (paper eq. 2):

$$\frac{1}{d} \approx \begin{cases} -0.000031\,d + 0.011002, & 128 \le d \\ -0.000122\,d + 0.022003, & 64 \le d < 128 \\ -0.000473\,d + 0.044006, & 32 \le d < 64 \\ -0.001862\,d + 0.088013, & 16 \le d < 32 \\ -0.007446\,d + 0.176041, & 8 \le d < 16 \\ -0.029800\,d + 0.352097, & 4 \le d < 8 \\ -0.119202\,d + 0.704193, & 2 \le d < 4 \\ -0.476822\,d + 1.408386, & d < 2 \end{cases} \tag{9}$$

Algorithm 1 Proposed online softmax

$m_0 \leftarrow -\infty$
$N \leftarrow \text{len}(\mathbf{x})$
$d \leftarrow 0$
for $i = 1$ **to** N **do**
    $m_i \leftarrow \text{IntMax}(m_{i-1},\, x_i)$
    $a \leftarrow d_{i-1} \times \text{shift-exp}(m_{i-1} - m_i)$
    $b \leftarrow \text{shift-exp}(x_i - m_i)$
    $d \leftarrow a + b$
end for
$r \leftarrow \text{pwl-recip}(d)$
for $j = 1$ **to** N **do**
    $f_j \leftarrow \text{shift-exp}(x_j - m_N)$
    $y_j \leftarrow f_j \times r$
end for

3 Simulation

3.1 Simulation Environment and Methodology

Every activation function was implemented twice: once as synthesisable Verilog, and once as a Python model. The Python model reproduces the hardware datapath exactly, using the same Q16.16 word, the same power-of-two slopes and segment boundaries. It is therefore a bit-accurate reference for the RTL rather than an idealised description of the algorithm.

Each unit was verified in two stages:

1. **Approximation accuracy.** The Python model is swept over the operating range of the function and compared against a float64 reference. The residual is reported as mean squared error and checked against the project requirement, MSE < 0.025.
2. **Standalone RTL verification.** The same model then emits the expected word for every swept input: as generated `check_result` calls pulled into the testbench with `` `include ``, or - for the vector-valued softmax - as paired `.mem` files read with `$readmemh`. A shared task compares each RTL output word against the model word with the four-state inequality `!==`, so a one-bit difference or an `X` is a failure.

3.2 Approximation Accuracy

Table 1 summarises stage one for every approximation in the design. The last column expresses the measured MSE as a percentage of the 0.025 budget allowed by the accuracy requirement, so it states directly how much of the error budget each unit consumes.

Table 1: Approximation error of each Q16.16 unit against a float64 reference. The requirement is MSE < 0.025.

Unit	Sweep (step)	Vectors	MSE	Budget used
Standalone activation units				
Sigmoid	$[-10, 10]$ (2^{-16})	655,373	4.70×10^{-5}	0.19 %
Tanh	$[-4, 4]$ (2^{-16})	524,289	2.35×10^{-4}	0.94 %
GELU	$[-5, 5]$ (2^{-16})	655,362	4.92×10^{-4}	1.97 %
ReLU	$[-8, 8]$ (2^{-15})	—	0	0 %
Softmax				
Submodule: shift-exp	$[-10, 0]$ (2^{-16})	655,361	1.55×10^{-3}	6.20 %
Submodule: PWL reciprocal	$[1, 256]$ (2^{-12})	1,044,481	4.94×10^{-6}	0.02 %
Full Softmax (avg.)	$N \leq 500$	1,050	1.03×10^{-4}	0.41 %
Full Softmax (worst)	$N \leq 500$	1,050	8.58×10^{-3}	34.3 %

The scalar sweeps are exhaustive over the range in which each function is non-trivial. Each unit is a pure function of a single 32-bit word, and outside the swept band the output is constant: the sigmoid saturates to 1.0 for $x \geq 5$, the tanh for $|x| \geq 2.5$, and shift-exp underflows to zero. Sweeping every representable input inside the band, and adding directed points outside it, therefore exercises every distinct behaviour of the module. Softmax operates on a vector and cannot be covered exhaustively.

The dominant error of the sigmoid is the linear fit rather than the number format: its largest deviation occurs in the widest segment ($2.375 \leq x < 5$, slope 1/32) and

exceeds the Q16.16 quantisation step of 1.5×10^{-5} by three orders of magnitude. The tanh is obtained from the same core through the identity $\tanh(x) = 2\sigma(2x) - 1$, and the measurement confirms that the identity introduces no additional rounding: the tanh MSE over $[-4, 4]$ equals four times the sigmoid MSE over the corresponding band $[-8, 8]$. The GELU is the least accurate scalar unit because its output scales a sigmoid error by x; its error is quoted against the ± 5.0 full scale, since a relative measure is undefined at $\mathrm{GELU}(0) = 0$. The ReLU introduces no approximation and its error is identically zero.

Two properties of the softmax primitives govern the accuracy of the complete engine. The shift-exp approximation is exact at $x = 0$ and returns 0.74998 one LSB below zero, where $\lfloor 1.5x \rfloor$ decrements while the mantissa term remains close to 1.5. The PWL reciprocal returns 0.9316 instead of 1.0 at $d = 1$, and repeats the same 6.8 % relative error at $d = 2, 4, 8, \ldots$, its segments being fitted one per octave. Both effects appear in the reconstructed output sums, which span 0.9316 to 1.72 rather than unity: a one-hot vector yields $d = 1$ and is scaled by 0.9316, while in Pass 1 the denominator is rescaled by shift-exp$(m_{\mathrm{prev}} - m_{\mathrm{new}})$ at every update of the running maximum, a factor of 0.75 instead of unity for arguments immediately below zero, so vectors in ascending order accumulate the deviation. Deviation from unity is consequently not used as a pass criterion; the MSE requirement is, and it is met.

3.3 Standalone RTL Verification

The sigmoid, tanh and GELU paths are verified by `sigmoid_subenv_tb.sv`, which drives all three functions through a single `check_result` task. The opcode is set before each vector `` `include ``, so the task accumulates a separate error count per function. Across 1,835,024 comparisons the RTL output matched the Python expectation bit-exactly for all three functions. The GELU path carries one more pipeline stage than the sigmoid and tanh paths and therefore returns its result one cycle later; the

testbench synchronises on `ready`. The sigmoid core is additionally verified in isolation by `sigmoid_tb.sv`. The ReLU is exact by construction and has no separate testbench.

The softmax primitives are verified by `shift_exp_tb.sv` and `pwl_reciprocal_tb.sv` against their 655,361 and 1,044,481 vector pairs, including the $v_{\mathrm{int}} = 0$ boundary, the underflow flush and the segment clamp at $d \geq 256$. Both are bit-exact. The complete engine is verified by `softmax_subenv_tb.sv`, which encloses it in a behavioural SRAM and drives it as the DLX does: the input vector is staged in memory, `start` is pulsed, and after `done` the written-back words are compared element by element against the golden output. Driver and checker are implemented as classes communicating over a sign-up queue, with a functional coverage collector recording the length and shape class of each case. The recorded run of the 110-vector set ($N = 2 \ldots 16$, 15 directed and 95 constrained-random) returned 108/110; the two failures are extreme-spread vectors, in which the output sum deviates to approximately 1.35 through the rescaling mechanism described in Section 3.2. The vector set has since been regenerated to the full 1050 cases with N up to 500, and the corresponding run is pending. Verification of the instruction decode, the control FSM and the memory arbitration between the softmax DMA and the CPU is described in Section 4.1.6, with the resulting measurements in Section 5.

4 Implementation

The project is implemented as synthesizable Verilog RTL: a base multi-cycle DLX processor extended with an `ActivationEngine` on the datapath, four new control-FSM states, and a memory arbiter that lets the softmax DMA share the external SRAM with the CPU. Figure 3 shows the complete system; orange blocks are the hardware *added* by this project, blue and purple blocks are the base DLX.

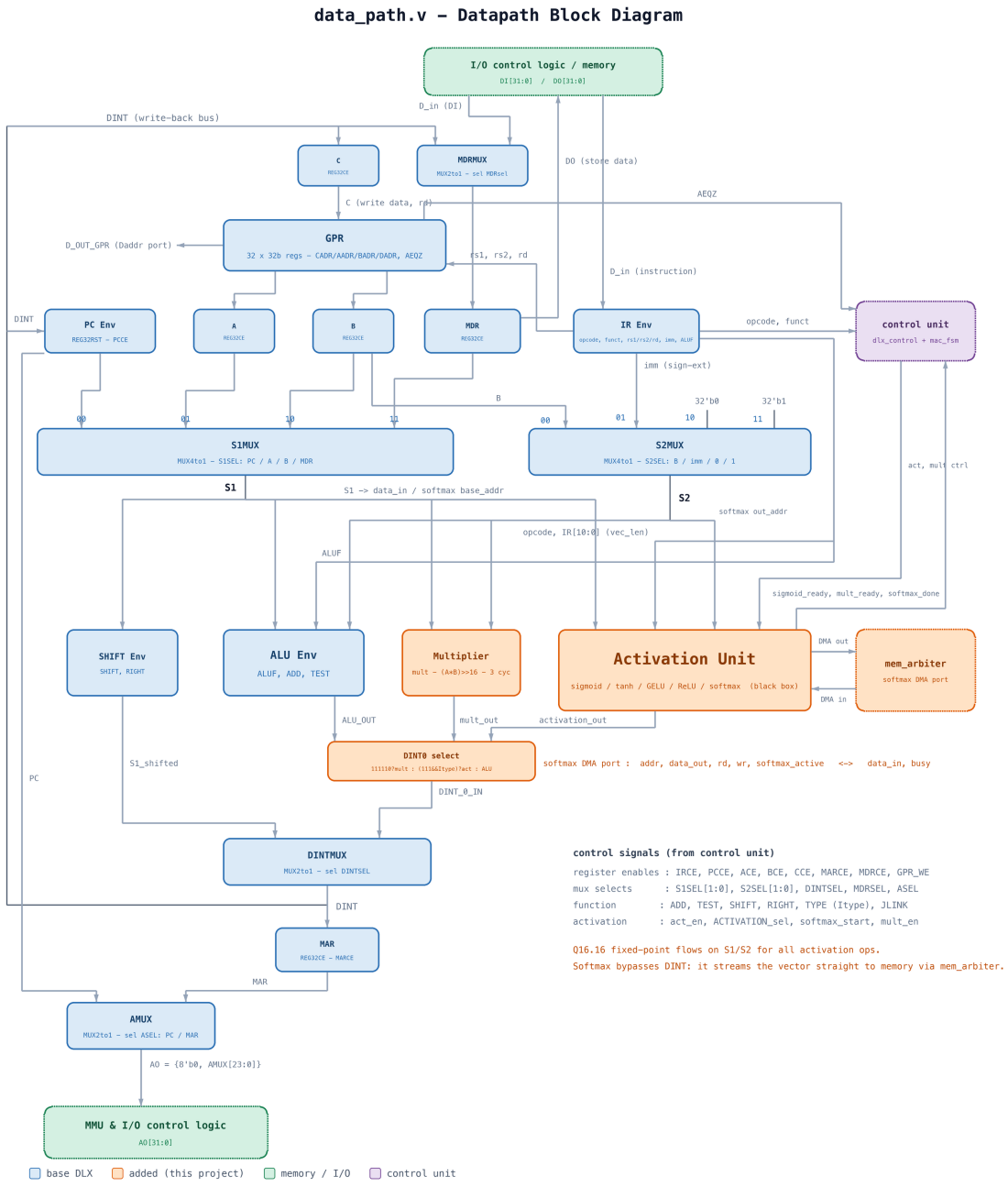

Figure 3: The extended datapath (`data_path.v`). Orange marks the hardware added by this project. The base registers, muxes and ALU/shifter (blue) are unchanged.

4.1 Hardware Description

4.1.1 Instruction-Set Extension

Five opcodes were added under the previously unused `111...` opcode space (Table 2). The four scalar instructions follow the I-type register-to-register pattern (input `rs1`, result `rd`); **softmax is Z-type**: a new added type, it consumes two source registers - `rs1` (instruction bits [25:21]) holds the input vector's base address, `rs2` (instruction bits [20:16]) the output base address - and takes the vector length from instruction bits [10:0], allowing $1 \leq N \leq 2047$. It writes no GPR; its result is the output vector in memory. The sigmoid group shares one decode family (D3, opcode `1110**`), with `opcode[1:0]` selecting the function inside the engine.

Table 2: New DLX instructions.

Instruction	Opcode [31:26]	Type	Operation
`sigmoid`	`111000`	I	$R_d = \sigma(R_{s1})$
`tanh`	`111001`	I	$R_d = \tanh(R_{s1})$
`gelu`	`111010`	I	$R_d = \mathrm{GELU}(R_{s1})$
`relu`	`111100`	I	$R_d = \max(0, R_{s1})$
`softmax`	`111101`	Z	$\mathrm{mem}[R_{s2}..] = \mathrm{softmax}(\mathrm{mem}[R_{s1}..]),\ N = \mathrm{IR}[10:0]$

4.1.2 `ActivationEngine.v` - Engine Top Level

`ActivationEngine.v` (new file) is a thin structural wrapper that instantiates the five functions (Figure 4). The scalar input `data_in` is the datapath's S1 operand bus; `ACTIVATION_sel` chooses between the sigmoid group and ReLU for the register write-back path, while the softmax unit is start/done-driven and owns its own memory port. The wrapper also carries the `activation_en` input and the `sigmoid_ready` output, which present the multi-cycle handshake of the pipelined sigmoid path to the control FSM.

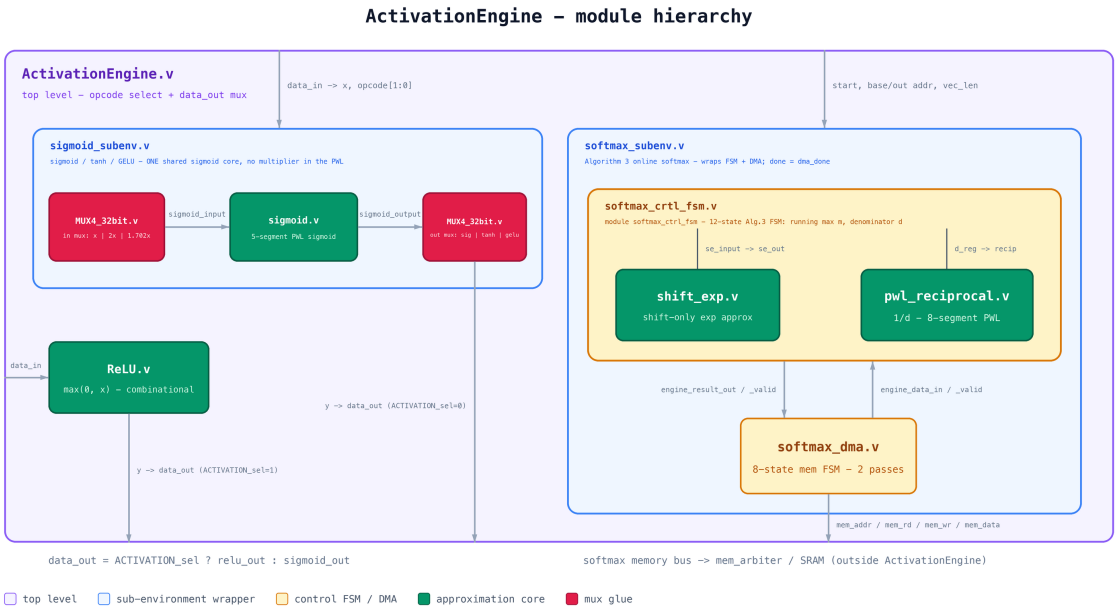

Figure 4: `ActivationEngine.v` module hierarchy.

4.1.3 `sigmoid_subenv` - One Shared Sigmoid Core

Three instructions (σ, tanh, GELU) would naively need three sigmoid evaluators; `sigmoid_subenv.v` (new file) instantiates *one* (Figure 5). An input mux (`MUX4_32bit`) selects the core's argument - x for sigmoid, $x \ll 1$ for tanh, or the $1.702x$ shift-add tree for GELU - and an output mux selects the post-processing: the raw core output for sigmoid, $(\cdot \ll 1) - 1$ for tanh, or the product $x \cdot \sigma$ for GELU.

The core itself (`sigmoid.v`, new file) implements Eq. (5): it takes $|x|$, selects one of five segments by magnitude comparison, computes $(|x| \ggg s) + b$ with $s \in \{2, 3, 5\}$, and applies the symmetry $1 - y$ for negative inputs. It contains no multiplier and no memory - comparators, one shifter and one adder.

The unit is pipelined, and the registered stages are drawn in yellow in Figure 5. The GELU path is cut by the register pair `x_g1`/`sig_g1`, which hold the multiplicand and the sigmoid result for one cycle so the multiply sits in a clock period of its own.

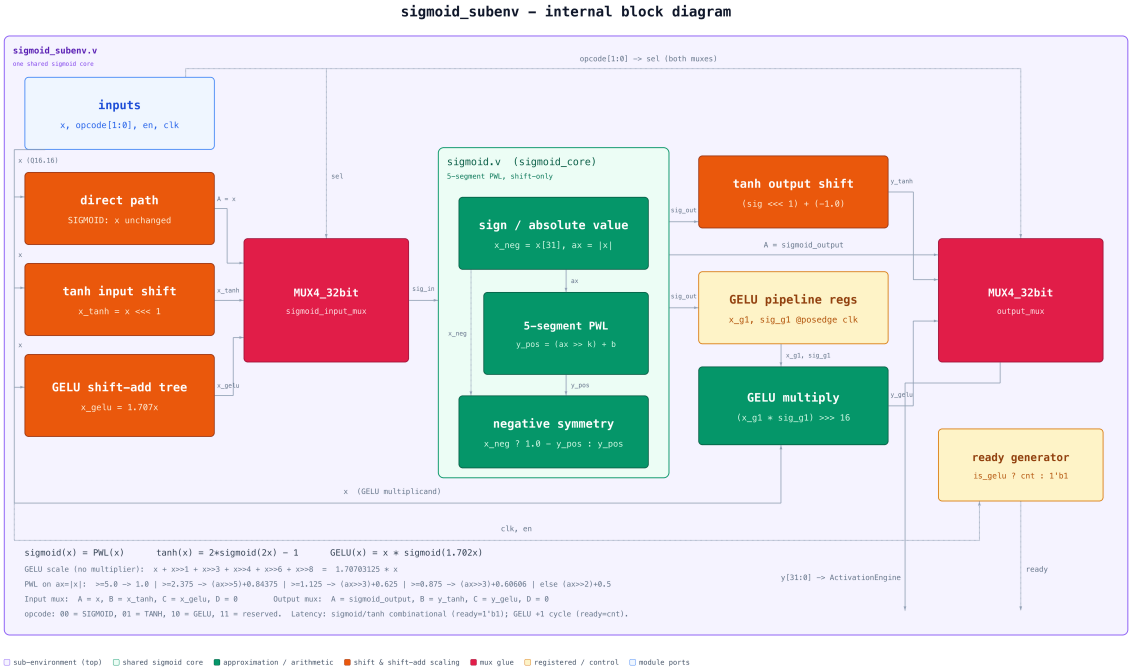

Figure 5: `sigmoid_subenv` internal block diagram.

4.1.4 `ReLU.v`

ReLU needs no approximation: in two's complement, $\max(0, x)$ is a sign-bit test. `ReLU.v` (new file) is a single-line combinational module - output zero when `x[31]` is set, else x - and completes within its execute state with no handshake.

4.1.5 Softmax Engine

Softmax is the largest new unit (Figure 6), composed of six new files:

- `softmax_subenv.v`: top-level wrapper binding the control FSM and DMA.
- `softmax_crtl_fsm.v`: a 12-state FSM executing Algorithm 1. It owns the m/d registers, one shared `shift_exp` instance, the `pwl_reciprocal` instance and one multiplier.

- `softmax_dma.v`: an 8-state FSM generating the sequential word addresses (base$+i$, matching the processor's word-addressed `lw`/`sw` model) and the read/write strobes for both passes, handshaking elements to the control FSM The input and output base addresses and the vector length are latched on the start pulse.
- `shift_exp.v` Eq. (8): one shift-add, a field splice and a variable shifter; flushes to zero when the shift amount exceeds 31. No multiplier.
- `pwl_reciprocal.v` Eq. (9): a priority encoder on d's integer part selects one of eight (a_k, b_k) pairs , followed by one multiply and one add.
- `mem_arbiter.v`: a combinational 2:1 mux over the full SRAM signal group. While `softmax_active` is high the DMA owns the address/data/strobe lines and the CPU-side `ack` is masked; otherwise the CPU path passes through untouched. The arbiter feeds the *existing* `mac_fsm`, so the SRAM handshake logic is reused rather than duplicated.

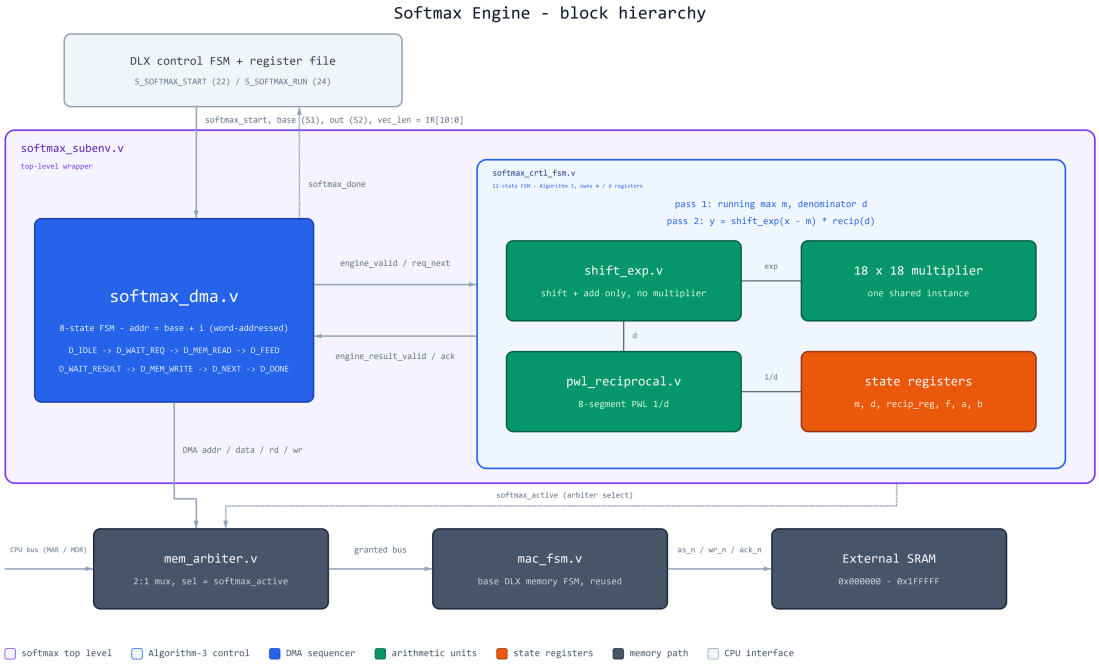

Figure 6: Softmax engine.

4.1.6 DLX Core Modifications

Four base-DLX files were modified to host the engine; one is reused untouched:

- `dlx_control.v` - the heart of the integration. The FSM grew from 21 to 25 states Figure 7): `S_SIGMOID` (21) drives the sigmoid group and, since the pipelining, loops on itself until `sigmoid_ready` before handing over to the I-type write-back state; `S_RELU` (23) is single-cycle; `S_SOFTMAX_START` (22) pulses `softmax_start` while the register file's S1/S2 ports present the two base addresses; `S_SOFTMAX_RUN` (24) spins until `softmax_done`. Three decode families were added on {opcode,funct}: D3 (`1110**`, sigmoid group), D10 (`111101`, softmax) and D11 (`111100`, ReLU). The

module also generates the new control outputs `sigmoid[1:0]`, `ACTIVATION_sel`, `act_en` and `softmax_start`.

- `data_path.v` - instantiates the `ActivationEngine`, feeds it S1 (scalar operand and softmax input base), S2 (softmax output base) and `IR[10:0]` (vector length), and extends the C register's write-back mux so activation results enter the register file through the standard path. The softmax memory port and `softmax_active` are routed to the arbiter.
- `DLX.v` / `control.v` - structural plumbing: wire the new control/status signals (`act_en`, `sigmoid_ready`, softmax start/done/active) between control and datapath, and instantiate the `mem_arbiter` between the CPU memory port and the SRAM pins.
- `mac_fsm.v` - **unchanged**. The DMA reaches the SRAM through the arbiter and the existing handshake FSM, which is the key reuse decision of the memory design.

4.2 Software Description

The project's software falls into two groups:

- **Golden models (Python)**: Every unit has a bit-accurate Q16.16 twin that reproduces the RTL truncations exactly, sweeps its input range against float64, and emits the test vectors that the testbenches read.
- **Assembly test programs**: Each activation has two programs: one computes it in software on the base instruction set, the other calls the new instruction. Both run the same approximation, so the comparison measures the hardware and not a change of method. Each pair reads the same input and writes to the same addresses, so the outputs match word for word and the cycle counts of Section 5 come from the same run.
- **Updating the RESA environment**: the RESA assembler builds `.cod` images from the instruction table in `commands.xml`, so we added the five new opcodes there, without them the accelerated programs will not assemble. `OPCODES.md` mirrors the same table and was updated alongside it.

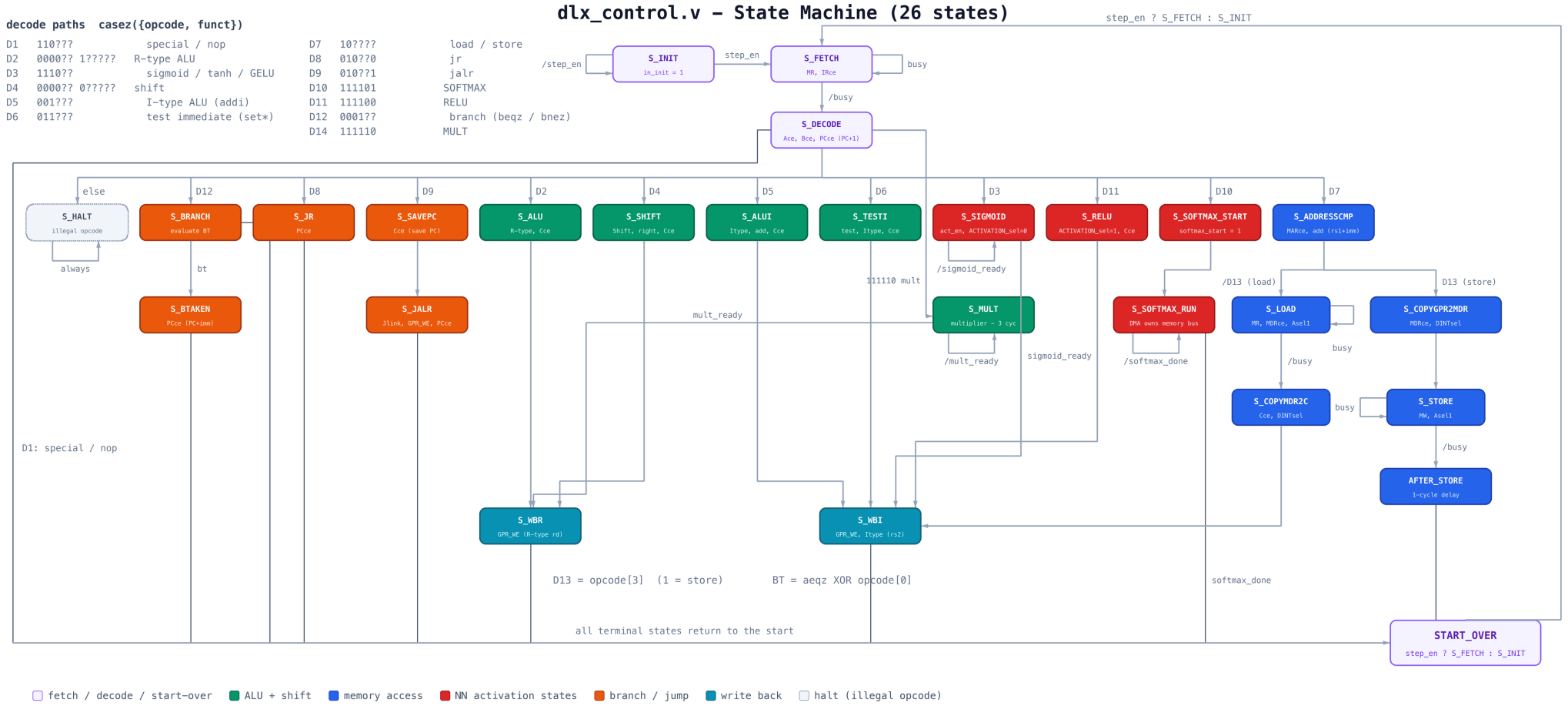

Figure 7: `dlx_control` FSM: the base instruction cycle together with the activation states (red).

5 Analysis of Results

This chapter compares the extended processor against its baselines along the project's three axes: Latency, accuracy and area.

5.1 Latency

Latency is measured in clock cycles per activation evaluation, comparing the hardware instruction against the equivalent software routine running on the same processor (the SW/HW assembly pairs of Section 4.2).

Expressed as the requirement states it, four of the five functions clear the 30% bar with wide margin and the combined figure is 93.2% (Figure 8). ReLU is the exception at 20.8%: it is a single sign-bit test, so the software routine is already only eight instructions and there is little for the instruction to remove. The saving grows with the cost of the function it replaces, which is why softmax, the most expensive, gains the most.

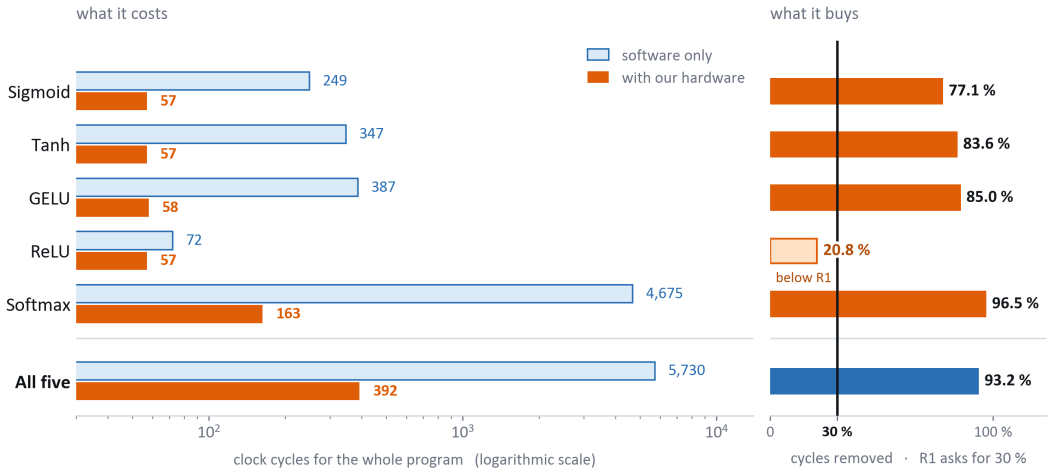

Figure 8: Clock cycles per demo program, software against the hardware activation path.

5.2 Accuracy

Simulation-side accuracy is complete (Section 3.2): all functions meet the MSE target with margin in the bit-exact golden-model sweeps.

Figure 9 places every function against the 0.025 budget. The worst case, GELU at 4.92×10^{-4}, is roughly fifty times inside it, and ReLU is exact by construction.

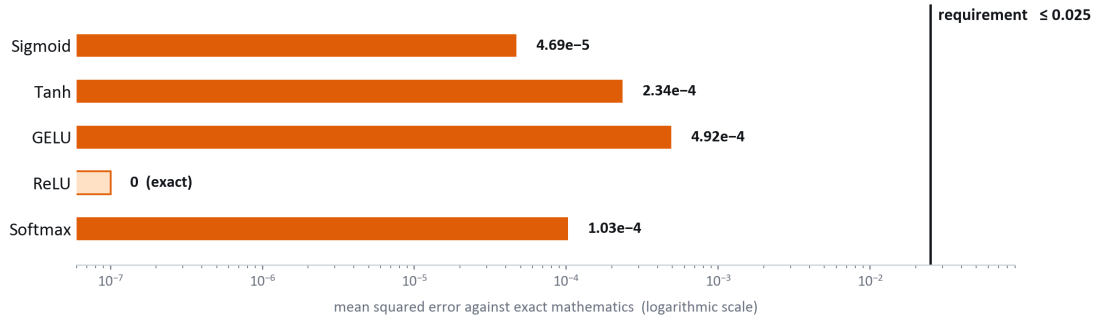

Figure 9: Mean squared error of each activation against exact mathematics, logarithmic scale, with the 0.025 requirement marked.

5.3 Area

Area is measured as post-map device utilization on the XC6SLX25-2, comparing the baseline processor against DLX-NN under the same ISE 14.7 settings. The activation path costs 1,073 → 2,282 slice LUTs and 676 → 1,183 slice registers. The budget is ≤ 20% of the baseline LUT count per activation function, so five functions are allowed 100% between them; the measured total is +112.7%, an average of 22.5% each. The aggregate therefore lands just over budget by 12.7 percentage points.

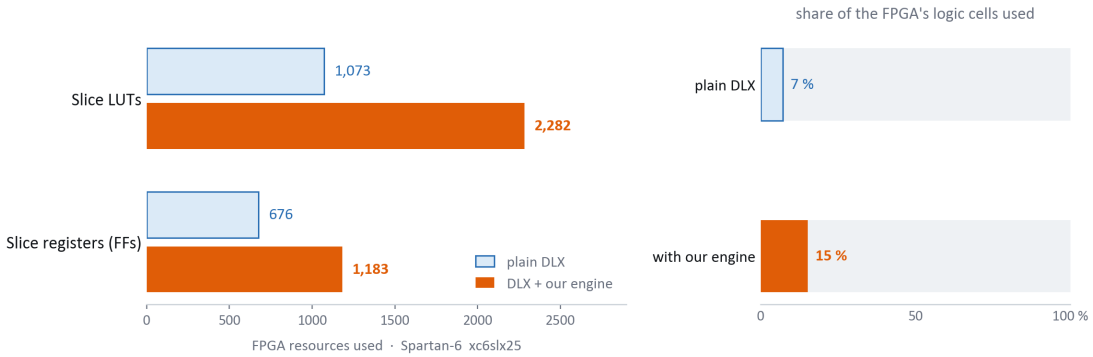

Figure 10: FPGA resources used, plain DLX against Extended DLX, and the share of the Spartan-6 xc6slx25 each occupies.

Almost all of it is softmax. The other four activations are scalar and combinational, sigmoid, tanh and GELU share a single shift-only core, and ReLU is a sign test, so they add a fixed slice of datapath and little else. Softmax is a different kind of instruction: it operates on a vector in memory rather than a register, which means it needs its own DMA engine to walk the data, a control FSM to sequence the two passes over it, and storage to carry the running maximum and denominator between them.

The trade is worth taking because the block that consumes the area is the block that returns the most time. Softmax is the most expensive function to emulate in software and the only one whose cost grows with vector length; it is also the one the hardware speeds up most, 4,675 → 163 cycles (Figure 8). Spending 2.13× the LUTs to buy 28.7× the softmax throughput is a 13.5× improvement in area–delay product, and still 6.87× taken over all five programs.

6 Conclusions and Further Work

6.1 Results vs. Goals

Measured against the goals of Section 1.1:

- **Functional goal → met.** The DLX executes five new activation instructions: decode, execute states, write-back and (for softmax) DMA over the shared SRAM.
- **Accuracy → met.** Every function clears the MSE < 0.025 bar with at least an order of magnitude of margin in full-range sweeps.
- **Latency → partially met.** We added five new functions. Four of them pass the 30% goal easily. ReLU does not, and that is expected: it is a single comparison against zero, so the software version is already only a few instructions long and there is almost nothing for hardware to save. Taken together, Figure **??** shows a 93.2% cycle reduction over the five functions combined, so the requirement holds at the network level even though ReLU alone misses it.
- **Area — missed, by a small margin.** We budgeted 20% extra LUTs per function, about 100% for all five. The implementation uses 2,282 slice LUTs against 1,073 for the base DLX, an overhead of 113%, or 22% per function. together.

6.2 Future Development Directions

- **Add a vector form of the scalar instructions.** Softmax already shows the payoff of amortizing decode over a whole vector: 96.5% fewer cycles against software, the best result in the project. A batched sigmoid or tanh over n elements would give the other four functions the same advantage, which matters as soon as a real layer rather than a single value is being computed.
- **Revisit the number format.** Q16.16 is generous for inference and the area result is the reason to question it. A Q8.8 datapath would roughly halve the register and multiplier cost, and the golden models already exist to quantify exactly how much accuracy that buys back.

7 Project Documentation

All project deliverables are version-controlled in a public GitHub repository:

`https://github.com/saleh2411/DLX-NN`

The repository is organised along the three implementation layers of the project, each verified bit-exact against the one above it: the Python golden models define the numerical reference, the Verilog RTL reproduces the same arithmetic in hardware, and the DLX assembly programs run that arithmetic on the processor itself.

Table 3: Repository structure and contents.

Location	Contents
`README.md`	ISA and opcode tables, approximation formulas, measured accuracy and testbench status, build commands and toolchain versions.
`py/`	Python golden models, the numerical reference for everything else: `sigmoid/` (`sigmoid.py`, `tanh.py`, `GELU.py`) and `Softmax/` (`softmax_golden.py` for Algorithm 3, `pwl_reciprocal.py`, `shift_exp.py`). Each script writes its own error summary and plots.
`design/`	Activation-engine RTL, one directory per unit: `ActivationEngine.v` (top-level opcode mux), `sigmoid/` (σ/tanh/GELU core), `relu/`, `softmax/` (control FSM, DMA, `shift_exp`, `pwl_reciprocal`, `mem_arbiter`) and `mult/`, with their testbenches, vector tables, Makefiles and the compile list `sources.f`.
`asm/`	Matched software-versus-hardware program pairs, one directory per activation: `<fn>_sw.s` uses only the base ISA, `<fn>_hw.s` the custom instruction, with the assembled `.cod` and a flat `.data` SRAM image. Both run identical math on identical inputs, so their cycle counts compare directly.
`asm/nn-demo/`	The end-to-end demonstration: a 16×16 handwritten-digit classifier ($256 \to 16 \to 10$, tanh hidden layer, softmax output). `py-nn/` trains and quantises it to Q16.16 through the golden models; `asm-nn/` emits it as DLX assembly and holds ready-to-run programs for digits 0–9 in both forms.
`DLX/`	The processor itself: `HOME_VER_before_edit/`, the simulation and ISE project with the activation engine and arbiter wired into `DLX.v`, `data_path.v` and `dlx_control.v` (Figure 7); `SOURCE_VER_before_edit/`, the FPGA source version with constraints file and bitstream; and the laboratory baselines `IO_SIMUL_VER/` and `Lab_base/`.
`RESA_ENV/`	RESA monitor working set: the assembler opcode table `commands.xml` (source of truth) and `OPCODES.md`, label files, `.cod` test programs and synthesised `.bit` bitstreams.
`Documents/`	This report, the final presentation and the results workbook.

8 References

[1] Z. Pan *et al.*, "A Modular Approximation Methodology for Efficient Fixed-Point Hardware Implementation of the Sigmoid Function," *IEEE Transactions on Industrial Electronics*, 2022.

[2] K. Hirayae *et al.*, "Hardware-Oriented and Precisely Approximated Online Softmax for Deep Learning Models," *IEEE International Conference on Electronics, Circuits and Systems (ICECS)*, 2025.

[3] "Simple DLX - Laboratory Specification and Handouts," Tel Aviv University, Faculty of Engineering.